Lab1: Datalab

郑欣宜

25300120165

```
duuugh@LAPTOP-T1CNGP6A:~/ICS_1_Datalab$  ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 9/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 12/56 operators
rotateRightBits: 12/16 operators
roundEvenPow2: 20/24 operators
midpointTowardFirst: 18/32 operators
isBetweenEitherOrder: 32/48 operators
mul5Sat: 30/30 operators
classifyAdd3: 27/52 operators
floatScaleThreeHalves: 34/60 operators
floatRoundEven: 25/65 operators
float_i2f: 20/40 operators
bitCount: 29/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
duuugh@LAPTOP-T1CNGP6A:~/ICS_1_Datalab$ ./btest
Score   Rating  Errors  Function
 1      1       0       signMask
 2      2       0       bitXor
 3      3       0       negativePart
 4      4       0       copyByteWithin
 4      4       0       logicalShift
 4      4       0       swapNibblePairs
 4      4       0       secondLowestZeroBit
 5      5       0       oddParity
 5      5       0       rotateRightBits
 5      5       0       roundEvenPow2
 5      5       0       midpointTowardFirst
 7      7       0       isBetweenEitherOrder
 7      7       0       mul5Sat
 7      7       0       classifyAdd3
 7      7       0       floatScaleThreeHalves
 10     10      0       floatRoundEven
 10     10      0       float_i2f
 10     10      0       bitCount
 10     10      0       bitReverse
Total points: 110/110
```

思路（bits.c 中每题写有注释，包含在此思路中）：

1. signMask

00000000 00000000 00000000 00000001 左移 31 位

2. bitXor

排除对应位同为 1 或同为 0 的情况，即是异或

3. negativePart

先利用算术右移生成选择掩码：当 x 非负时，x>>31 为全 0；当 x 为负时，结果为全 1。再用 ~x+1 求补码相反数，将两者按位与，便能在非负时返回 0，在负数时保留相反数。

4. copyByteWithin

用 src<<3 和 dst<<3 将字节编号转换为位移量。先将源字节右移到最低 8 位，再与 0xFF 按位与，得到高位全为 0 的 copySrc。然后构造 ~(0xFF<<dstShift)，清除目标字节，最后将源字节移到目标位置并按位或合并。helpRemove 是目标 8 位全为 0，其他位全为 1，这样才能清除目标字节并保留其他字节。

5. logicalShift

算术右移 x>>n 会在高位补符号位，而逻辑右移要求高位补 0。因此构造掩码 ~(((1<<31)>>n)<<1)，使其高 n 位为 0、低 32-n 位为 1，再与算术右移结果按位与，清除符号扩展产生的高位。

6. swapNibblePairs

先构造掩码 0x0F0F0F0F，标记每个字节的低 4 位。(x & shift)<<4 将各字节的低 4 位移到高 4 位；(x>>4) & shift 将高 4 位移到低 4 位，并清除右移带来的符号扩展。两部分占据的位置互不重叠，所以通过 A+B 合并就能完成交换，此处加法与按位或效果相同。

7.secondLowestZeroBit

先令 y=~x，把寻找第二个 0 转换为寻找第二个 1。y+~0 相当于 y-1：减去 1 会使最低的 1 变为 0，其右侧的 0 变为 1；再计算 y & (y-1)，就能清除最低的 1，得到 removeLowest。最后用 removeLowest & (~removeLowest+1) 提取剩余的最低一个 1，它对应原数第二个最低的 0。如果原数不足两个 0，结果自然为 0。

8.oddParity

位异或是奇函数，有奇数个 1 结果为 1，与题目要求相反。x ^ (x>>16)是高 16 位与低 16 位一一对应做异或，结果的低 16 位即为所求。再依次右移 8、4、2、1 位进行异或，将异或结果对半合并，最终得到 1 位答案，即为 32 位相异或的结果。最后用 !(x&1) 将结果反转。

9. rotateRightBits

先用 shift=n & 31 将移位量限制在 0～31，对非负 n 相当于 n%32。再计算 leftShift=(~shift+1) & 31：当 shift>0 时等于 32-shift，当 shift=0 时等于 0，避免移动 32 位。用掩码将 x>>shift 转为逻辑右移，得到保留在右侧的部分；用 x<<leftShift 得到移回左侧的部分，最后按位或合并。

10. roundEvenPow2

令 element=1<<n 表示相邻倍数之间的间距，用全 1 的 mask 左移 n 位后清除 x 的低 n 位，得到不大于 x 的最近倍数 below。计算 distance=x-below，并与半个间距 half 比较：diff=half-distance 为负时，说明更接近上方倍数；为 0 时，说明恰好在中间。belowOdd=(below>>n)&1 判断下方倍数对应的商是否为奇数。更接近上方，或恰好居中且下方商为奇数时，令 chooseAbove=1，返回 below+(1<<n)；否则返回 below。这样就实现了中点时选择偶数倍。

11. midpointTowardFirst

直接计算 x+y 可能溢出，因此利用数学关系。different 中的 1 标记 x,y 只有一方为 1 的位。

x&y 中的 1 标记 x,y 皆为 1 的位，即相加时发生进位的位；由于 x+y=2(x&y)+x^y，中点向下取整 lower=x&y+different>>1（表示 x^y 除以 2 向下取整）。signX：x 为负时全 1，否则全 0。signDiff：x,y 异号时全 1，同号时全 0。如果 x,y 异号，greaterMask=~signX，若 x>y 则全 1，若 x<y 则全 0。如果 x,y 同号，同号相减不会溢出，greaterMask=difference>>31。different & greaterMask & 1 的结果是 1 或 0，只有在和为奇数，而且 x>y 时才为 1。

12. isBetweenEitherOrder

不必先确定 a、b 的大小，而是分别判断 x<a 和 x<b。比较时，若两数异号，就看 x 的符号；若同号，就看差值的符号，从而避免用可能溢出的异号差值判断大小。signA 在 x、a 异号时为全 1，同号时为全 0；compareA 在 x<a 时为全 1，否则为全 0，compareB 同理。两次比较结果不同时，说明 x 已处于区间范围内；再用 !(x^a) 和 !(x^b) 补上等于任一端点的情况，实现包含端点的区间判断。

13. mul5Sat

先确定哪些输入乘以 5 不会溢出。通过构造 0x33333333 再右移一位，得到上界 max=0x19999999=429496729，取相反数得到下界 min=-429496729。signNeg 在 x 为负时全为 1；结合 distanceNeg=x-min 的符号，可以判断负向溢出，得到 negOverflow。同样，结合非负掩码与 distancePos=max-x 的符号判断正向溢出。随后构造 INT_MIN、INT_MAX 和 normalMask=~(negOverflow | posOverflow)：负向溢出选择 INT_MIN，正向溢出选择 INT_MAX，未溢出则选择 (x<<2)+x，即 5x。

14. classifyAdd3

直接相加可能溢出，因此将每个数拆成商和余数：x=4*(x>>2)+(x&3)：(1)x>>2 即除以 4，既能保证和不溢出（-2^31<=x,y,z<=2^31-1, -2^29<=(x>>2),(y>>2),(z>>2)<=2^29-1, INT_MIN<-3*2^29<3*2^29-3<INT_MAX），又能用算数右移实现；(2)x&3 表示 x 除以 4 的余数，原理是余数是最低两位，以 3=0011 为 mask 取 x 的最低两位。其中，算术右移两位得到除以 4 向下取整的商，x&3 取最低两位，得到 0～3 的非负余数。三个商相加得到 high，范围仍在 int 内；三个余数相加得到 low，再用 low>>2 将凑够的 4 进到高部分，得到 totalHigh。边界可以写成 \(INT_MIN=4(-2^{29})+0\) 和 \(INT_MAX=4(2^{29}-1)+3\)，因此只需比较 totalHigh 与 min=-2^29、max=2^29-1。distanceMin=totalHigh-min 为负时返回 -1；distanceMax=max-totalHigh 为负时返回 1；否则返回 0。

15. floatScaleThreeHalves

浮点数的位表示（科学计数法）：1 位符号位 s+8 位指数位 exp+23 位小数位 frac（正规化数 1<=M<2，frac=M-1)。NaN 和正负无穷都原样返回。正规化数不回省略的 1，0 不需要补。shift 默认为 1，因为题目是除以 2。realFrac>2 时，需要额外右移 1 位使 1<=realFrac<2，同时指数加 1。if(result>=0x800000)：非规格化数可能变为正规格化数；else if(result>=0x1000000)：舍入可能使有效数再次进位。if(exp>=0xFF)：指数溢出，返回对应符号的无穷。return s | (exp<<23) | (result & 0x7FFFFF)：去掉隐含位，拼回三个字段

16. floatRoundEve

先分割符号、指数和小数字段。实际指数为 E=exp-127。 当 exp>=150 时，有限数的实际指数至少为 23，已经没有小数位，直接返回；NaN 和无穷也由这个分支原样返回。当 exp<126 时，绝对值小于 0.5，返回带原符号的零；当 exp==126 时，frac==0 表示恰好为 ±0.5，舍入为 ±0，否则舍入为 ±1.0。剩余情况中，shift=150-exp=23-E 表示有效数中对应数值小数部分的位数，1u 表示无符号常量 1。用 mask 提取这些低位为 remainder，其表示的小数大小为 remainder/2^shift，所以 half=1u<<(shift-1) 对应 0.5。先通过 uf & ~mask 清除小数位，再在超过一半，或恰好一半且整数部分为奇数时，给位模式增加 1u<<shift，使数值的绝对值增加 1。

17. float_i2f

输入为 0 时直接返回 0。其他情况先提取符号，再将输入赋给无符号变量 magnitude，负数通过无符号的 ~magnitude+1 求大小，从而也能处理 INT_MIN。将指数初始化为 158=127+31，不断左移 magnitude，直到最高的 1 到达第 31 位，每左移一次便将指数减 1。此时第 31 位作为隐含的 1，第 30～8 位作为 23 位尾数，最低 8 位用于舍入。余数大于 0x80 时向上舍入，等于 0x80 时仅在尾数最低位为 1 时进位。最后使用加法组合符号、指数和尾数，使尾数舍入产生的进位能够自动进入指数。

18. bitCount

先构造 0x55555555、0x33333333 和 0x0F0F0F0F 三个掩码，分别用于提取每 2 位组中的低 1 位、每 4 位组中的低 2 位和每 8 位组中的低 4 位。先将相邻单个位相加，得到每 2 位中 1 的个数；再合并相邻计数，得到每 4 位、每 8 位中的计数。之后通过 x+(x>>8) 和 x+(x>>16) 将四个字节的计数汇总到低位。总数范围为 0～32，用 x & 0x3F 保留最低 6 位即可得到答案。

19. bitReverse

先构造 mask16=0x0000FFFF，再通过异或和左移依次生成 mask8=0x00FF00FF、mask4=0x0F0F0F0F、mask2=0x33333333 和 mask1=0x55555555。利用这些掩码，依次交换相邻的 1 位、2 位、4 位和 8 位分组，最后交换高低 16 位。每一步都把一组右移、另一组左移，再按位或合并；右移后使用掩码清除符号扩展产生的高位，保证负数也能正确处理。经过这些分组交换，原来第 i 位就被移到第 31-i 位，即可。